**February 6th:** Code created and developed to sort unstructured data notes into time bins, ultimately this was not needed as we only took the last note, but was relevant in February when we were thinking about using multiple notes for 1 patient

```
# ========================
# Mount Google Drive
# ========================
from google.colab import drive
drive.mount("/content/drive", force_remount=True)

# ========================
# Imports
# ========================
import pandas as pd
import numpy as np

# ========================
# Parameters
# ========================
BIN_HOURS = 6
N_BINS = 3
WINDOW_HOURS = BIN_HOURS * N_BINS  # 18 hours

# ========================
# Load embeddings with charttime
# ========================
emb_path = "/content/drive/My
Drive/train_small_embeddings_bioclincalbert_with_charttime.csv"
df = pd.read_csv(emb_path)

# ========================
# Clean & convert types
# ========================
df["charttime"] = pd.to_datetime(df["charttime"], errors="coerce")
df["hadm_id"] = pd.to_numeric(df["hadm_id"], errors="coerce").astype("Int64")
df["subject_id"] = pd.to_numeric(df["subject_id"], errors="coerce").astype("Int64")

# Drop rows with invalid time
df = df.dropna(subset=["charttime"])

# ========================
# Reference time per admission
# ========================
df["ref_time"] = df.groupby("hadm_id")["charttime"].transform("max")
```

```
# Hours before reference
df["delta_h"] = (df["ref_time"] - df["charttime"]).dt.total_seconds() / 3600

# ==========================
# KEEP ONLY last 18 hours
# ==========================
df = df[(df["delta_h"] >= 0) & (df["delta_h"] <= WINDOW_HOURS)].copy()

# ==========================
# Assign time bins
# ==========================
# delta_h in:
#   [0,6)   -> bin 3 (most recent)
#   [6,12)  -> bin 2
#   [12,18] -> bin 1

bin_raw = np.floor(df["delta_h"] / BIN_HOURS).astype(int)
df["time_bin"] = (N_BINS - bin_raw).astype(int)

# ==========================
# Final cleanup & save
# ==========================
df = df.sort_values(["hadm_id", "charttime"]).reset_index(drop=True)

out_path = "/content/drive/My Drive/train_small_embeddings_bioclinicalbert_18hr_binned.csv"
df.to_csv(out_path, index=False)

# ==========================
# Sanity checks
# ==========================
print("Saved to:", out_path)
print("Total rows kept:", len(df))
print(df[["hadm_id", "charttime", "delta_h", "time_bin"]].head(10))
```

**February 22nd:** Continuing working on sorting time bin embeddings and then combing unstructured and structured data

Step 1) Bin Level Sanity Check

```
# ==============================
# 1) Mount Google Drive
# ==============================
from google.colab import drive
drive.mount('/content/drive')

# ==============================
# 2) Set file path (EDIT this path if needed)
# ==============================
embedding_path =
"/content/drive/MyDrive/train_small_embeddings_bioclinicalbert_18hr_binned.csv"

# If your file is inside a folder, example:
# embedding_path =
"/content/drive/MyDrive/BME499/train_small_embeddings_bioclinicalbert_18hr_binned.csv"

# ==============================
# 3) Load embeddings file
# ==============================
import pandas as pd

E = pd.read_csv(embedding_path)

print("Columns:")
print(E.columns.tolist())

print("\nTotal rows (notes):", len(E))

# ==============================
# 4) Check bin structure
# ==============================
if "hadm_id" in E.columns and "time_bin" in E.columns:
    print("\nUnique hadm_id:", E["hadm_id"].nunique())
    print("Unique time_bin values:", sorted(E["time_bin"].unique()))

    print("\nNotes per (hadm_id, time_bin):")
    print(E.groupby(["hadm_id","time_bin"]).size().describe())
else:
    print("\n⚠️ Could not find hadm_id and/or time_bin columns.")
```

Output:
Total rows (notes): 6120

Unique hadm_id: 2318
Unique time_bin values: [np.int64(1), np.int64(2), np.int64(3)]

Notes per (hadm_id, time_bin):
count  3911.000000
mean      1.564817
std       0.903718
min       1.000000
25%       1.000000
50%       1.000000
75%       2.000000
max      11.000000
dtype: float64

Step 2) Combine Vectors

```
# ========================================
# Load both files from Google Drive
# ========================================
from google.colab import drive
drive.mount('/content/drive')

import pandas as pd
import re

structured_path =
"/content/drive/MyDrive/train1_all_features_table_18hrs.csv"
embedding_path  =
"/content/drive/MyDrive/train_small_embeddings_bioclinicalbert_18hr_binned
.csv"

S = pd.read_csv(structured_path)
E = pd.read_csv(embedding_path)

# Standardize column names
S = S.rename(columns={"HADM_ID":"hadm_id", "SUBJECT_ID":"subject_id"})
E = E.rename(columns={"HADM_ID":"hadm_id", "SUBJECT_ID":"subject_id"})

# ========================================
```

```
# 1) Separate structured into static + bin columns
# ========================================
key_cols = [c for c in ["hadm_id","subject_id"] if c in S.columns]
struct_cols = [c for c in S.columns if c not in key_cols]

bin_pattern = re.compile(r"bin([123])\b", re.IGNORECASE)
binned_cols = [c for c in struct_cols if bin_pattern.search(c)]
static_cols = [c for c in struct_cols if c not in binned_cols]

# ========================================
# 2) Convert structured into LONG format (per bin)
# ========================================
S_long_list = []

for b in [1,2,3]:
   cols_b = [c for c in binned_cols if re.search(fr"bin{b}\b", c,
flags=re.IGNORECASE)]
   rename_dict = {c: re.sub(fr"_?bin{b}\b", "", c, flags=re.IGNORECASE)
for c in cols_b}

   temp = S[key_cols + static_cols + cols_b].copy()
   temp = temp.rename(columns=rename_dict)
   temp["time_bin"] = b

   S_long_list.append(temp)

S_long = pd.concat(S_long_list, ignore_index=True)

# ========================================
# 3) Merge NOTE-LEVEL embeddings with correct bin structured features
# ========================================
note_level = E.merge(S_long, on=["hadm_id","time_bin"], how="inner")

print("Original notes:", len(E))
print("Merged notes:", len(note_level))

note_level.to_csv("/content/drive/MyDrive/train_note_level_struct_plus_emb
eddings.csv", index=False)
print("Saved merged note-level dataset.")
```

**March 2nd:** After speaking with care providers radiology notes were removed so those needed to be screened out

**Mount:**

```
from google.colab import drive
drive.mount('/content/drive')

import pandas as pd
import numpy as np
```

**Loud Cell CSVs:**

```
notes_path = "/content/drive/My Drive/note_with_dx_updated.csv"
notes_df = pd.read_csv(notes_path)

print("Total notes before filtering:", len(notes_df))
notes_df.head()
```

**Load Cell CSVs:**

```
rain_ids_path = "/content/drive/My Drive/trainingset_2.csv"
train_ids_df = pd.read_csv(train_ids_path)

print("Training set HADM_ID count:", len(train_ids_df))
train_ids_df.head()
```

**Filter Cell:**

```
# Lowercase column names (safe)
notes_df.columns = notes_df.columns.str.lower()
train_ids_df.columns = train_ids_df.columns.str.lower()

# Coerce hadm_id to numeric; anything like "hadm_id" becomes NaN
notes_df['hadm_id'] = pd.to_numeric(notes_df['hadm_id'], errors='coerce')
train_ids_df['hadm_id'] = pd.to_numeric(train_ids_df['hadm_id'], errors='coerce')

# Drop rows where hadm_id is missing (these are the bad header rows)
notes_df = notes_df.dropna(subset=['hadm_id'])
train_ids_df = train_ids_df.dropna(subset=['hadm_id'])

# Convert to int AFTER cleaning
notes_df['hadm_id'] = notes_df['hadm_id'].astype(int)
train_ids_df['hadm_id'] = train_ids_df['hadm_id'].astype(int)
```

```
# Filter
filtered_notes_df = notes_df[notes_df['hadm_id'].isin(train_ids_df['hadm_id'])]

print("Total notes after filtering:", len(filtered_notes_df))
print("Unique hadm_id after filtering:", filtered_notes_df['hadm_id'].nunique())
```

**Cell Save CSV:**

```
output_path = "/content/drive/My Drive/train_notes_filtered.csv"
filtered_notes_df.to_csv(output_path, index=False)

print("File saved to:", output_path)
```

**Cell Filtering Out Radiology Notes:**

```
from google.colab import drive
drive.mount('/content/drive')

import pandas as pd

base_folder = "/content/drive/My Drive/"
input_path = base_folder + "train_notes_filtered.csv"

# Load file
df = pd.read_csv(input_path)
print("Original rows:", len(df))

# Remove Radiology notes (case-insensitive)
df_no_rad = df[df["category"].str.lower() != "Radiology"].copy()

print("Rows after removing Radiology:", len(df_no_rad))

# Save cleaned file
output_path = base_folder + "train_notes_no_radiology.csv"
df_no_rad.to_csv(output_path, index=False)

print("Saved to:", output_path)
```

**March 9th:** Notes were filtered out from radiology, after getting feedback from GC and Selena started working on continuing to combine with new datasets from Selenda filtered into 12 hour time bins, no radiology notes

Sort Notes Prior to Embedding:

```
import pandas as pd
from google.colab import drive

drive.mount('/content/drive')

input_path = "/content/drive/My Drive/train_notes_no_radiology.csv"
output_path = "/content/drive/My Drive/train_notes_no_radiology_latest_per_hadm.csv"

df = pd.read_csv(input_path, low_memory=False)

# Clean column names
df.columns = df.columns.str.strip().str.lower()

print("Columns:", df.columns.tolist())
print("Original rows:", len(df))

# Convert charttime to datetime
df["charttime"] = pd.to_datetime(df["charttime"], errors="coerce")

# Drop rows where charttime is missing
df = df.dropna(subset=["charttime"])

# Keep latest note per hadm_id
idx = df.groupby("hadm_id")["charttime"].idxmax()
df_latest = df.loc[idx].reset_index(drop=True)

print("Rows after filtering:", len(df_latest))
print("Unique hadm_id:", df_latest["hadm_id"].nunique())

# Save
df_latest.to_csv(output_path, index=False)

print("Saved to:", output_path)
```

Embed Only latest note:

```
# If needed, uncomment these the first time:
# !pip install -q transformers torch sentencepiece

import os
import gc
import numpy as np
import pandas as pd
import torch
from tqdm.auto import tqdm
from transformers import AutoTokenizer, AutoModel
from google.colab import drive

# ==========================
# 1) Mount Google Drive
# ==========================
drive.mount('/content/drive')

# ==========================
# 2) File paths
# ==========================
input_path = "/content/drive/My Drive/train_notes_no_radiology_latest_per_hadm.csv"
output_path = "/content/drive/My
Drive/train_notes_no_radiology_latest_per_hadm_embeddings_bioclinicalbert.csv"

# ==========================
# 3) Load input CSV
# ==========================
df = pd.read_csv(input_path, low_memory=False)
df.columns = df.columns.str.strip().str.lower()

print("Input rows:", len(df))
print("Columns:", df.columns.tolist())

# Try to find the note text column automatically
possible_text_cols = ["text", "note", "note_text", "notes"]
text_col = None
for col in possible_text_cols:
    if col in df.columns:
        text_col = col
```

```
        break

if text_col is None:
    raise ValueError(f"Could not find a note text column. Available columns: {df.columns.tolist()}")

if "hadm_id" not in df.columns:
    raise ValueError("Column 'hadm_id' not found in the CSV.")

# Clean text
df[text_col] = df[text_col].fillna("").astype(str)

# ========================
# 4) Resume logic
#    If output already exists, skip completed hadm_ids
# ========================
if os.path.exists(output_path):
    done_df = pd.read_csv(output_path, usecols=["hadm_id"], low_memory=False)
    done_ids = set(done_df["hadm_id"].astype(str))
    before = len(df)
    df = df[~df["hadm_id"].astype(str).isin(done_ids)].copy()
    print(f"Resuming: skipped {before - len(df)} already embedded rows")
else:
    print("No existing output found. Starting fresh.")

print("Rows left to embed:", len(df))

if len(df) == 0:
    print("Nothing left to do. Output already complete.")
else:
    # ========================
    # 5) Load BioClinicalBERT
    # ========================
    model_name = "emilyalsentzer/Bio_ClinicalBERT"
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print("Using device:", device)

    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModel.from_pretrained(model_name).to(device)
    model.eval()

    # ========================
    # 6) Mean pooling function
    # ========================
    def mean_pool(last_hidden_state, attention_mask):
```

```
    mask = attention_mask.unsqueeze(-1).expand(last_hidden_state.size()).float()
    masked_embeddings = last_hidden_state * mask
    summed = masked_embeddings.sum(dim=1)
    counts = mask.sum(dim=1).clamp(min=1e-9)
    return summed / counts

# =========================
# 7) Encode a batch of texts
# =========================
def encode_batch(texts, max_length=512):
    encoded = tokenizer(
        texts,
        padding=True,
        truncation=True,
        max_length=max_length,
        return_tensors="pt"
    )

    encoded = {k: v.to(device) for k, v in encoded.items()}

    with torch.no_grad():
        outputs = model(**encoded)
        embeddings = mean_pool(outputs.last_hidden_state, encoded["attention_mask"])

    return embeddings.cpu().numpy()

# =========================
# 8) Batch process + append to CSV
# =========================
batch_size = 16   # lower this to 8 if you get GPU/RAM issues
write_header = not os.path.exists(output_path)

for start in tqdm(range(0, len(df), batch_size), desc="Embedding batches"):
    end = min(start + batch_size, len(df))
    batch_df = df.iloc[start:end].copy()

    texts = batch_df[text_col].tolist()
    batch_embeddings = encode_batch(texts)

    # Make embedding columns: emb_0 ... emb_767
    emb_df = pd.DataFrame(
        batch_embeddings,
        columns=[f"emb_{i}" for i in range(batch_embeddings.shape[1])]
    )
```

```
        batch_out = pd.concat(
            [batch_df.reset_index(drop=True), emb_df.reset_index(drop=True)],
            axis=1
        )

        # Append to output as we go
        batch_out.to_csv(
            output_path,
            mode="a",
            header=write_header,
            index=False
        )
        write_header = False

        # cleanup
        del batch_df, texts, batch_embeddings, emb_df, batch_out
        gc.collect()
        if torch.cuda.is_available():
            torch.cuda.empty_cache()

    print("Done.")
    print("Saved to:", output_path)
```

Get rid of columns that aren’t embeddings:

**April 20th:** After getting feedback from Selena and GC about the dimensions of the CSV files the old processed and combined datasets needed to be fixed to adjust for appropriate column and row numbers, code was written to find the discrepancy

Fixing missing columns

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

import pandas as pd

# ---- FILE PATHS (EDIT THESE IF NEEDED) ----
full_path = '/content/drive/MyDrive/train4_all_features_table_12hrs.csv'
set3_path = '/content/drive/MyDrive/set4_combined_12hr.csv'

# ---- LOAD DATA ----
df_full = pd.read_csv(full_path)
df_set3 = pd.read_csv(set3_path)

# ---- STANDARDIZE COLUMN NAMES ----
df_full.rename(columns={'HADM_ID': 'hadm_id'}, inplace=True)

# ---- ENSURE SAME TYPE ----
df_full['hadm_id'] = df_full['hadm_id'].astype(int)
df_set3['hadm_id'] = df_set3['hadm_id'].astype(int)

# ---- FIND MISSING IDs ----
full_ids = set(df_full['hadm_id'])
set3_ids = set(df_set3['hadm_id'])

missing_ids = full_ids - set3_ids

print("Total in full dataset:", len(full_ids))
print("Total in set3:", len(set3_ids))
print("Missing IDs:", len(missing_ids))

# ---- SAVE JUST THE MISSING IDS ----
missing_df = pd.DataFrame({'hadm_id': list(missing_ids)})
```

```
missing_df.to_csv('/content/drive/MyDrive/missing_hadm_ids4.csv',
index=False)

# ---- OPTIONAL: SAVE FULL ROWS FROM ORIGINAL DATA ----
missing_rows = df_full[df_full['hadm_id'].isin(missing_ids)]
missing_rows.to_csv('/content/drive/MyDrive/missing_full_rows4.csv',
index=False)

print("Files saved to Google Drive!")
```

**Final Google Colab Code Used to Process all Training Data Sets for Final Device:**

**Cell 1: Filter for Training Set Against Large Batch of Data**

```
import pandas as pd
from google.colab import drive

# 1. Mount Google Drive
drive.mount('/content/drive')

# 2. File paths (EDIT if needed)
base_path = "/content/drive/My Drive/"
notes_path = base_path + "notes_train_full.csv"
trainset_path = base_path + "trainingset_3.csv"

# 3. Load CSVs
notes_df = pd.read_csv(notes_path)
train_df = pd.read_csv(trainset_path)

# 4. Standardize column names (lowercase)
notes_df.columns = notes_df.columns.str.lower()
train_df.columns = train_df.columns.str.lower()

# 5. Ensure matching data types + clean formatting
notes_df['hadm_id'] = notes_df['hadm_id'].astype(str).str.strip()
train_df['hadm_id'] = train_df['hadm_id'].astype(str).str.strip()

# 6. Filter notes to only matching HADM_IDs
filtered_notes = notes_df[notes_df['hadm_id'].isin(train_df['hadm_id'])]

# 7. Save result
output_path = base_path + "notes_filtered_by_trainingset3.csv"
filtered_notes.to_csv(output_path, index=False)

# 8. Print summary
print("Original notes:", len(notes_df))
print("Training set HADM_IDs:", len(train_df))
print("Filtered notes:", len(filtered_notes))
print("Saved to:", output_path)
```

**Cell 2: Filter Out Radiology Notes and Embed with BioClinicalBert**

```
import pandas as pd
from google.colab import drive

# Mount Drive
drive.mount('/content/drive')

# Path
base_path = "/content/drive/My Drive/"
input_path = base_path + "notes_filtered_by_trainingset3.csv"

# Load
df = pd.read_csv(input_path)

# Standardize column names (just in case)
df.columns = df.columns.str.lower()

# Remove radiology notes
df_no_rad = df[df['category'].str.lower() != 'radiology']

# Save
output_path = base_path + "notes_filtered_no_radiology.csv"
df_no_rad.to_csv(output_path, index=False)

# Print results
print("Original rows:", len(df))
print("After removing Radiology:", len(df_no_rad))
print("Removed rows:", len(df) - len(df_no_rad))
print("Saved to:", output_path)

#Embed:
# ============================================================
# STEP 0: Install packages if needed
# Run this once in Colab if transformers/torch are missing
# ============================================================
# !pip install -q transformers torch sentencepiece tqdm

# ============================================================
# STEP 1: Imports + mount Google Drive
# ============================================================
import os
import gc
```

```
import numpy as np
import pandas as pd
import torch
from tqdm.auto import tqdm
from transformers import AutoTokenizer, AutoModel
from google.colab import drive

drive.mount('/content/drive')

# ============================================================
# STEP 2: File paths
# ============================================================
base_path = "/content/drive/My Drive/"

input_notes_path = base_path + "notes_filtered_no_radiology.csv"
latest_notes_path = base_path + "notes_filtered_no_radiology_latest_per_hadm.csv"
embeddings_output_path = base_path +
"notes_filtered_no_radiology_latest_per_hadm_embeddings_bioclinicalbert.csv"

# ============================================================
# STEP 3: Load CSV and keep only latest note per hadm_id
# ============================================================
df = pd.read_csv(input_notes_path, low_memory=False)

# Clean column names
df.columns = df.columns.str.strip().str.lower()

print("Columns:", df.columns.tolist())
print("Original rows:", len(df))

# Safety checks
if "hadm_id" not in df.columns:
    raise ValueError("Column 'hadm_id' not found.")
if "charttime" not in df.columns:
    raise ValueError("Column 'charttime' not found.")

# Convert charttime to datetime
df["charttime"] = pd.to_datetime(df["charttime"], errors="coerce")

# Drop rows where hadm_id or charttime is missing
df = df.dropna(subset=["hadm_id", "charttime"]).copy()

# Keep latest note per hadm_id
idx = df.groupby("hadm_id")["charttime"].idxmax()
```

```
df_latest = df.loc[idx].reset_index(drop=True)

print("Rows after keeping latest note per hadm_id:", len(df_latest))
print("Unique hadm_id:", df_latest["hadm_id"].nunique())

# Save latest-note-only file
df_latest.to_csv(latest_notes_path, index=False)
print("Saved latest-note file to:", latest_notes_path)

# ============================================================
# STEP 4: Find text column
# ============================================================
possible_text_cols = ["text", "note", "note_text", "notes"]
text_col = None

for col in possible_text_cols:
    if col in df_latest.columns:
        text_col = col
        break

if text_col is None:
    raise ValueError(f"Could not find a note text column. Available columns:
{df_latest.columns.tolist()}")

df_latest[text_col] = df_latest[text_col].fillna("").astype(str)
df_latest["hadm_id"] = df_latest["hadm_id"].astype(str)

print("Using text column:", text_col)

# ============================================================
# STEP 5: Resume logic
# If embeddings file already exists, skip already completed hadm_id rows
# ============================================================
if os.path.exists(embeddings_output_path):
    done_df = pd.read_csv(embeddings_output_path, usecols=["hadm_id"], low_memory=False)
    done_ids = set(done_df["hadm_id"].astype(str))

    before = len(df_latest)
    df_to_embed = df_latest[~df_latest["hadm_id"].isin(done_ids)].copy()
    print(f"Resuming: skipped {before - len(df_to_embed)} already embedded rows")
else:
    df_to_embed = df_latest.copy()
    print("No existing embeddings file found. Starting fresh.")
```

```
print("Rows left to embed:", len(df_to_embed))

# ============================================================
# STEP 6: Stop early if already complete
# ============================================================
if len(df_to_embed) == 0:
    print("Nothing left to embed. Output already complete.")
else:
    # ========================================================
    # STEP 7: Load BioClinicalBERT
    # ========================================================
    model_name = "emilyalsentzer/Bio_ClinicalBERT"
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print("Using device:", device)

    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModel.from_pretrained(model_name).to(device)
    model.eval()

    # ========================================================
    # STEP 8: Mean pooling
    # ========================================================
    def mean_pool(last_hidden_state, attention_mask):
        mask = attention_mask.unsqueeze(-1).expand(last_hidden_state.size()).float()
        masked_embeddings = last_hidden_state * mask
        summed = masked_embeddings.sum(dim=1)
        counts = mask.sum(dim=1).clamp(min=1e-9)
        return summed / counts

    # ========================================================
    # STEP 9: Batch encoder
    # ========================================================
    def encode_batch(texts, max_length=512):
        encoded = tokenizer(
            texts,
            padding=True,
            truncation=True,
            max_length=max_length,
            return_tensors="pt"
        )

        encoded = {k: v.to(device) for k, v in encoded.items()}

        with torch.no_grad():
```

```
        outputs = model(**encoded)
        embeddings = mean_pool(outputs.last_hidden_state, encoded["attention_mask"])

    return embeddings.cpu().numpy()

# ================================================
# STEP 10: Embed in batches and append to CSV
# ================================================
batch_size = 16   # change to 8 if you hit memory issues
write_header = not os.path.exists(embeddings_output_path)

for start in tqdm(range(0, len(df_to_embed), batch_size), desc="Embedding batches"):
    end = min(start + batch_size, len(df_to_embed))
    batch_df = df_to_embed.iloc[start:end].copy()

    texts = batch_df[text_col].tolist()
    batch_embeddings = encode_batch(texts)

    emb_df = pd.DataFrame(
        batch_embeddings,
        columns=[f"emb_{i}" for i in range(batch_embeddings.shape[1])]
    )

    batch_out = pd.concat(
        [batch_df.reset_index(drop=True), emb_df.reset_index(drop=True)],
        axis=1
    )

    batch_out.to_csv(
        embeddings_output_path,
        mode="a",
        header=write_header,
        index=False
    )
    write_header = False

    del batch_df, texts, batch_embeddings, emb_df, batch_out
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

print("Done embedding.")
print("Saved embeddings to:", embeddings_output_path)
```

**Cell 3: Combine Structured and Unstructured**

```
import pandas as pd
from google.colab import drive

# ====================================
# 1) Mount Google Drive
# ====================================
drive.mount('/content/drive')

# ====================================
# 2) File paths
# ====================================
base_path = "/content/drive/My Drive/"

embeddings_path = base_path +
"notes_filtered_no_radiology_latest_per_hadm_embeddings_bioclinicalbert.csv"
structured_path = base_path + "train3_all_features_table_12hrs.csv"
output_path = base_path + "train3_structured_plus_bioclinicalbert_embeddings.csv"

# ====================================
# 3) Load CSVs
# ====================================
emb_df = pd.read_csv(embeddings_path, low_memory=False)
struct_df = pd.read_csv(structured_path, low_memory=False)

# ====================================
# 4) Clean column names
# ====================================
emb_df.columns = emb_df.columns.str.strip().str.lower()
struct_df.columns = struct_df.columns.str.strip().str.lower()

print("Embeddings columns:", emb_df.columns.tolist()[:20], "...")
print("Structured columns:", struct_df.columns.tolist()[:20], "...")

# ====================================
# 5) Make sure both have hadm_id
# ====================================
if "hadm_id" not in emb_df.columns:
    raise ValueError("Column 'hadm_id' not found in embeddings CSV.")

if "hadm_id" not in struct_df.columns:
    raise ValueError("Column 'hadm_id' not found in structured CSV after lowercasing.")
```

```
# Convert to same type
emb_df["hadm_id"] = emb_df["hadm_id"].astype(str).str.strip()
struct_df["hadm_id"] = struct_df["hadm_id"].astype(str).str.strip()

# ====================================
# 6) Safety: keep only one row per hadm_id
# ====================================
# Embeddings file should already be one row per hadm_id,
# but this keeps the first just in case
emb_df = emb_df.drop_duplicates(subset=["hadm_id"]).copy()

# Structured file should also be one row per hadm_id
struct_df = struct_df.drop_duplicates(subset=["hadm_id"]).copy()

print("Unique hadm_id in embeddings:", emb_df["hadm_id"].nunique())
print("Unique hadm_id in structured:", struct_df["hadm_id"].nunique())

# ====================================
# 7) Optional: remove note metadata columns before merging
#    Keep hadm_id + embedding columns only
# ====================================
embedding_cols = ["hadm_id"] + [col for col in emb_df.columns if col.startswith("emb_")]
emb_only_df = emb_df[embedding_cols].copy()

print("Number of embedding columns:", len([c for c in emb_only_df.columns if
c.startswith("emb_")]))

# ====================================
# 8) Merge by hadm_id
# ====================================
merged_df = struct_df.merge(emb_only_df, on="hadm_id", how="inner")

# ====================================
# 9) Save merged file
# ====================================
merged_df.to_csv(output_path, index=False)

# ====================================
# 10) Print summary
# ====================================
print("Rows in structured file:", len(struct_df))
print("Rows in embeddings file:", len(emb_only_df))
print("Rows in merged file:", len(merged_df))
```

```
print("Columns in merged file:", len(merged_df.columns))
print("Saved to:", output_path)
```